

A Simple Tutorial on On-Policy and Off-Policy Learning

What Does “Policy” Mean?

A policy is simply the mechanism that decides what action or output to produce. For a language model, we write the policy as

$$\pi_{\theta}(y \mid x),$$

- x is the input or prompt,
- y is a generated response,
- θ represents the model parameters,
- $\pi_{\theta}(y \mid x)$ is the probability that the model generates response y given prompt x .

In the context of LLM training, *policy* can simply be understood as **the model that generates the responses**.

On-Policy Learning

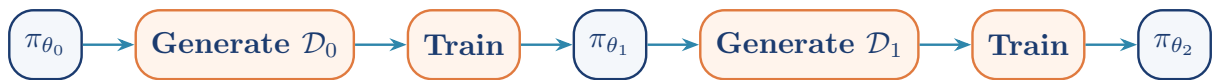
Basic Idea

In **on-policy learning**, the training examples are generated by the **current version of the model**. Suppose the current model is π_{θ_0} . We give it a prompt x and ask it to generate several candidate responses:

$$y_1, y_2, \dots, y_N \sim \pi_{\theta_0}(\cdot \mid x).$$

These responses are scored, ranked, or otherwise evaluated, and the selected responses are used to update the same model: $\pi_{\theta_0} \rightarrow \pi_{\theta_1}$.

The important point is that, in the next training round, we generate **new** responses using the **new** model π_{θ_1} , and the process repeats.



The generator continually follows the latest model.

Simple Example

Prompt: What is 15×4 ?

The input is the prompt $x = \text{"What is } 15 \times 4\text{"}$. The current model π_{θ_t} generates four candidate answers:

$$y_1 = 60, \quad y_2 = 55, \quad y_3 = 60, \quad y_4 = 45.$$

Where the scorer sits: immediately after generation, before training. A scorer or evaluator (rule-based checker, reward model, or human) assigns each (x, y_i) pair a score:

$$r_1 = 1, \quad r_2 = 0, \quad r_3 = 1, \quad r_4 = 0$$

where $r_i = 1$ denotes a correct answer (i.e. $y_i = 60$).

What the training data looks like: the stored records combine the prompt, the model's own answer, and the score:

$$\{(x, y_1, r_1), (x, y_2, r_2), (x, y_3, r_3), (x, y_4, r_4)\}.$$

This batch — generated by π_{θ_t} itself and scored immediately — is what gets used to update the model, producing $\pi_{\theta_{t+1}}$.

For the *next* iteration we do not reuse these four answers — we sample fresh ones from the updated model, and the scorer evaluates *those* instead:

$$y'_1, \dots, y'_N \sim \pi_{\theta_{t+1}}.$$

Key idea: on-policy means the training data is generated using the model's **current** policy.

Why Does On-Policy Training Matter?

As the model learns, the types of mistakes it makes also change. An early model may frequently produce very poor answers; after several updates those simple mistakes disappear, and the model instead makes more subtle mistakes. With on-policy training, we continuously generate new samples from the latest model:

$$\pi_{\theta_0} \rightarrow \pi_{\theta_1} \rightarrow \pi_{\theta_2} \rightarrow \dots \quad \text{and correspondingly} \quad \mathcal{D}_0 \rightarrow \mathcal{D}_1 \rightarrow \mathcal{D}_2 \rightarrow \dots.$$

This keeps the training process focused on the weaknesses of the **current** model.

Off-Policy Learning

Basic Idea

In **off-policy learning**, the responses used for training were generated by a policy **different** from the current model. For example, an older model π_{θ_0} or another model π_B generates a dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^M$, which is stored and later used to train a newer model π_{θ_5} . Since $\mathcal{D}$ was not generated by π_{θ_5} , the training is off-policy.



The data-generating policy does not necessarily change together with the model being optimized.

Why Use Off-Policy Training?

Generating new model outputs at every optimization step can be expensive. Off-policy learning lets us reuse previously generated data:

$$\mathcal{D} = \{(x_i, y_i, r_i)\}_{i=1}^M$$

can be generated once and reused for many gradient updates, substantially reducing generation cost. The trade-off is that the stored samples may no longer accurately represent the behavior of the current model.